Data Structure Rationale

ICT283 Assignment 2
Prasanna Kunwar 35744248

1. Overview of the chosen design

The program stores the weather readings in a map of binary search trees. There is one map entry for each month, and each entry holds a binary search tree of the readings taken during that month. The map is keyed by an integer that combines the year and the month, for example 201406 for June 2014, and each tree is keyed by a hashed date and time. In code this is a single member: `Map<int, Bst<KeyedRecord> > m_months`.

This section explains why the two required structures — the template BST with function-pointer traversals and the map — are used in this particular arrangement, together with the supporting Vector class. For each structure it gives the reasoning, the advantages of using it this way, the drawbacks that come with the choice, and the alternative arrangements that would also have worked.

The design was guided by one observation about the data and the menu. Every menu option asks a question about either a single month or a single year. Option 1 is about a month, option 2 walks the twelve months of a year, option 3 looks at one month across all years, and option 4 writes a whole year. Because a month is the unit every option is built around, a month is the natural thing to key the outer structure on.

2. The map (outer structure)

Usage and reasoning

The map holds one entry per month, so looking up a month is a single, direct operation and looking up a year is twelve look-ups that sit next to one another. The map keeps its keys in sorted order, which matters for two of the menu options: option 2 needs the months of a year in calendar order, and it gets that for free because the keys 201401, 201402 and so on already come out in order. The map's value type is a whole binary search tree, so each month carries its own tree of readings rather than the readings being mixed together in one large structure.

The class used is my own `Map`, a small template that wraps `std::map`. The wrapper exposes only four operations — `Insert`, `Find`, `Size` and `ForEach` — and `std::map` appears in that one header and nowhere else in the program. `ForEach` walks the entries in key order and hands each one back through a function pointer, so the map never has to expose an iterator and the rest of the program never has to know that a standard-library map is underneath. `Find` copies the value out into a reference parameter rather than returning a reference to the stored value, so a caller cannot reach in and change what the map holds.

Pros of this usage

– A month look-up is fast, and a year is simply twelve adjacent look-ups.
– The keys are always sorted, so the calendar order needed by option 2 comes for free.
– Wrapping `std::map` keeps the standard library out of the rest of the code, so the program depends on a small, clearly named interface instead of on map iterators.

– Copying values out of `Find` protects the stored data from being changed by accident.

Cons of this usage

– Splitting the readings into one tree per month means the readings are grouped by month, so an option that needed to range across month boundaries would have to visit several trees. None of the four options needs that, but the design does commit to the month being the main unit.
– Option 3 has to walk the map to find the matching month in each year, and that walk is the main cost of this arrangement. With only about fifty months in total the walk is far too small to measure, but it is still a cost that a different key choice could avoid.
– The wrapper adds a copy on `Find`. For a whole tree that copy is not free, so the loading path is written to build each month's tree once and insert it once, rather than reading a tree out, changing it and putting it back.

Alternatives that could have worked

– **A single BST of all readings, no map.** Every reading could sit in one tree keyed by date and time. This is simpler to describe, but every menu option would then have to walk the whole tree and test each reading's month or year, turning cheap look-ups into full traversals.
– **A map keyed by year, holding a tree of a whole year.** This would make option 4 (a whole year) a single look-up, but option 1 and option 3, which are about a single month, would then have to filter a year's worth of readings. Keying on the month suits three of the four options; keying on the year suits only one.
– **A map of Vectors instead of a map of trees.** A vector per month would also hold the readings, but the assignment requires the template BST with function pointers to be used in the processing, so the tree is the correct inner structure here.

3. The binary search tree (inner structure)

Usage and reasoning

Each month's readings are held in a template binary search tree. The tree stores `KeyedRecord` values and orders them by a key. The tree does no input or output of its own; instead its three traversals each take a function pointer (`void (*Visit)(const KeyedRecord&)`) and call that routine once for each reading. The client decides what happens to each reading — a statistics routine gathers the values it needs, and the file routine collects the values for one line of output. This keeps the tree general: it knows how to walk and hand back readings, but nothing about weather or about what is being calculated.

The reading is handed to the call-back as a `const` reference on purpose. If a client could change a reading in place it could change the part used for ordering, and the reading would then be sitting in the wrong place in the tree with nothing to warn anyone. The `const` stops that and keeps the tree's ordering rule safe.

The key and the hashing decision

The readings arrive in time order. If the tree were keyed by a plain increasing date-and-time value, each new reading would always be larger than the last, so every reading would go down the right-hand side and the tree would grow into a long line rather than a balanced shape. A tree in that state is no better than a list to search, and building it can use so much memory for the nested calls that the program stops. To avoid this, the key is passed through a radix (base) conversion before it is inserted: the digits of the date-and-time number are treated as the factors of a polynomial with a prime base, which mixes the digits so that readings taken one after another end up with values that are nowhere near each other. They then spread across the tree instead of going down one side. The tree class itself is not changed at all; only the number handed to it is.

The base of the conversion has to be larger than any digit it works on. Decimal digits run from 0 to 9, so the base must be at least 10, and a prime is chosen so the mixing is more even. Base 11, the smallest prime above 9, is used. This choice was tested directly against the data. With base 11 all 208,891 readings from the six supplied files are stored, with none lost. Lowering the base to 7 — which is below 10 and so cannot tell the higher digits apart — causes many different readings to hash to the same key; the tree then treats them as duplicates and keeps only one, and the stored total falls to 139,536, a loss of about a third of the data. Base 11 is therefore not an arbitrary choice: it is the point at which the hashing stops losing readings. The keys are held in `long long` rather than `long`, because the combined date-and-time value is large enough to overflow a 32-bit integer and wrap negative, which would corrupt the ordering.

Pros of this usage

– The function-pointer traversals keep the tree free of any input or output and free of any knowledge of weather, so the same tree could store anything with an ordering.
– Searching a month's readings does not require testing every reading, because the tree is ordered.
– Hashing the key keeps the tree from degenerating into a list on time-ordered data, so the ordering is genuinely useful; all 208,891 readings load with none lost.
– Because the tree hands data back through a call-back, the statistics and file routines can reuse the same traversal for different purposes.

Cons of this usage

– A binary search tree is not self-balancing, so its shape depends entirely on the order the keys arrive in. The hashing addresses this for this data, but the tree gives no guarantee on its own.
– Hashing the key means the readings inside a tree are no longer in time order. None of the four options needs them in time order — they all aggregate rather than list — but it is a property that has been given up.
– A function pointer cannot carry state, so a call-back that needs to gather results has to use file-scope variables to collect into. This works and is contained in one place, but it is less tidy than a method that could hold its own state.

Alternatives that could have worked

– **A self-balancing tree (such as an AVL or red-black tree).** This would guarantee a shallow tree regardless of key order and remove the need to hash the key. It was not used because the assignment requires the template BST from the labs, and because hashing the key solved the depth problem without changing the tree.
– **Keeping the plain date-time key and accepting the deep tree.** This is simplest to write but was rejected because the readings arrive in time order, which would make a plain-keyed tree behave like a list.
– **A hash table instead of a tree.** A hash table would give fast look-up of a single reading, but it would not keep the readings in any order and the assignment specifically requires the ordered BST with function pointers.

4. The Vector (supporting structure)

Usage and reasoning

Wherever the program needs a simple growable list — the list of data-file names read from `data_source.txt`, the list of readings passed from the reader to the store, and the lists of values gathered for the statistics routines — it uses my own `Vector` class rather than a raw array or `std::vector`. The statistics routines (`Average`, `SampleStandardDeviation`, `mad`, `sPCC`) depend only on `Vector<float>` and know nothing about weather, so they could be reused for any list of numbers.

Pros, cons and alternative

Pros: the Vector grows as needed, so no fixed size limit has to be guessed in advance, and it checks its own bounds, which a raw array does not. Keeping the standard library wrapped means the rest of the program depends on one small, known interface.
Cons: the wrapper is a thin layer over what `std::vector` already does, so it repeats work the standard library provides; the benefit is control and the requirement to encapsulate, not new capability.
Alternative: `std::vector` could have been used directly and would work, but the assignment requires every standard container except `std::map` to be wrapped in an own class for marks to be given, so wrapping it in `Vector` is the correct choice here.

5. Summary

The map of trees keyed by month follows the shape of the menu: three of the four options ask about a month, so the month is the key, and the fourth (a whole year) is twelve adjacent look-ups. The binary search tree with function-pointer traversals keeps the data handling separate from the data structure, and hashing its key keeps it shallow on time-ordered data. The Vector wraps the remaining list needs so that only `std::map` is used directly. The main cost accepted is the walk in option 3 and the commitment to the month as the central unit; both were judged worthwhile because they keep the design simple to explain and correct for every menu option, which the earlier labs stressed as the first priority.